

Definição do Trabalho Final

A definição do trabalho consiste em implementar uma aplicação que simula o funcionamento de uma rede local em anel. A aplicação deverá implementar a transmissão de mensagens entre as máquinas que compõem o anel, utilizando o protocolo UDP como transporte. Deve ser implementada uma fila para armazenar as mensagens que serão enviados por cada máquina da rede, e somente um item da fila (mensagem) pode ser transmitido por vez.

Em uma rede em anel, há o uso de *tokens*, que são pacotes especiais que circulam na rede e permitem a transmissão das mensagens por cada máquina da rede. Desta forma, a aplicação deverá implementar este *token* que ficará circulando na rede.

A topologia em anel deverá ser construída a partir de uma lista ordenada de máquinas ativas. Cada máquina deverá identificar seu sucessor como o próximo elemento da lista, considerando comportamento circular (o último elemento conecta-se ao primeiro).

O programa deverá possuir três tipos de pacotes: **controle**, *token* e **dados**.

Arquivo de inicialização

Ao iniciar o programa, o usuário deverá informar um apelido (letras A, B, C...), o tempo do *token* e dos dados e a probabilidade para inserir erro nas mensagens. Tais informações devem ser inseridas na aplicação através de um arquivo de configuração.

O arquivo de configuração deverá seguir o seguinte formato:

```
<apelido_da_máquina_atual>
<tempo_token_e_dados>
<probabilidade para inserir erro nas mensagens>
<timeout do token>
<tempo mínimo entre tokens>
```

Exemplo:

```
B
2
20
2,5
2
```

Inicialização do anel

Ao iniciar o programa, a aplicação deve enviar uma mensagem de **DISCOVER** (em broadcast), na **porta 6000**, se identificando. Quando uma máquina receber uma mensagem de DISCOVER, ela precisa responder com **HELLO** (em broadcast), também se identificando.

Exemplo com 4 máquinas (A, B, C e D). Todas as aplicações começam a rodar e todas enviam:

DISCOVER A IP_A

DISCOVER B IP_B

DISCOVER C IP_C

DISCOVER D IP_D

Todas respondem:

HELLO A IP_A

HELLO B IP_B

HELLO C IP_C

HELLO D IP_D

O anel deve ser formado em ordem alfabética, como no exemplo a seguir:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

A última máquina deve se ligar na primeira.

A primeira máquina (A) gera o token a primeira vez.

Funcionamento do anel

As máquinas devem enviar tokens e dados para a próxima máquina do anel (ordem alfabética de apelido).

A máquina que gera o *token* pela primeira vez deve enviá-lo para a próxima máquina do anel. Caso a máquina que recebeu o *token* não tenha dados para transmitir (fila de mensagens vazia), o *token* será enviado para a próxima máquina do anel. Caso contrário, a primeira mensagem é retirada da fila e é enviada para a próxima máquina.

Os dados enviados deverão retornar à máquina origem e somente depois disso o *token* poderá ser enviado para a próxima máquina.

Quando a máquina origem enviar um pacote de dados, um campo no cabeçalho do pacote deverá ser marcado como “maquinainexistente”. Esse pacote poderá retornar para a máquina origem com uma das seguintes configurações:

- “**maquinainexistente**”: significa que a máquina destino não se encontra na rede ou está desligada. Neste caso, uma **mensagem na tela** deve informar o ocorrido, a mensagem deve ser retirada da fila e o *token* deve ser transmitido para a próxima máquina do anel;
- “**NAK**”: significa que a máquina destino identificou um erro no pacote e o mesmo deverá ser retransmitido pela origem. Neste caso, uma **mensagem na tela** deve informar o ocorrido, o *token* deve ser transmitido para a próxima máquina do anel, e a mensagem não deve ser retirada da fila, sendo retransmitida na próxima passagem do *token*;
- “**ACK**”: significa que o pacote foi recebido corretamente pela máquina destino. Neste caso, uma **mensagem na tela** deve informar o ocorrido, a mensagem deve ser retirada da fila e o *token* deve ser transmitido para a próxima máquina do anel.

Módulo de inserção de falhas

Antes de enviar uma mensagem, o cálculo de controle de erro deverá ser inserido na mensagem pela máquina origem. Deve-se utilizar o **CRC32** como técnica de controle de erro. A aplicação deve implementar um **módulo de inserção de falhas** que force as máquinas a inserirem erros aleatoriamente nas mensagens. Este módulo deve trabalhar com alguma probabilidade para inserir erro nas mensagens, que deve ser inserido no arquivo de configuração.

Ao receber uma mensagem, a máquina destino deve recalcular o controle de erro. Caso a mensagem tenha sido recebida com erro, ela deve ser marcada com “**NAK**”, caso contrário ela deve ser marcada com “**ACK**”.

Fila de mensagens

Deverá ser implementada uma **fila de mensagens** em cada máquina. Esta fila poderá estar vazia ou não. A fila poderá conter **até 10 mensagens**. Para cada mensagem adicionada, deve ser armazenado também o apelido da máquina destino.

Formas de envio

Os serviços de envio de dados oferecidos pela aplicação devem contemplar duas formas de transmissão:

- **Unicast**: envia o pacote para um único destino;
- **Broadcast**: envia o pacote para todas as máquinas da rede usando o apelido **BROADCAST**. Neste caso, o módulo de inserção de falhas deve manter o pacote em “**maquinainexistente**”.

Descrição dos Pacotes

A implementação da aplicação deve seguir fielmente o formato dos pacotes descritos a seguir, pois durante a apresentação do trabalho, **aplicações de grupos diferentes deverão se comunicar**. As corretas interações entre as diferentes implementações fazem parte da avaliação do trabalho.

DISCOVER

O DISCOVER será formado por uma sequência numérica em formato string e terá o valor 10. Neste caso, o valor 10 será seguido de um ‘:’ e pelos campos:

<apelido da origem>:<endereço IP da origem>

Exemplo:

```
10:A:10.32.143.20
```

HELLO

O HELLO será formado por uma sequência numérica em formato string e terá o valor 20. Neste caso, o valor 20 será seguido de um ':' e pelos campos:

<apelido da origem>:<endereço IP da origem>

Exemplo:

```
20:B:10.32.143.21
```

Token

O *token* será formado por uma sequência numérica em formato string e terá o valor 1000, como mostra o exemplo a seguir:

```
1000
```

Pacote de dados

Um pacote de dados é formado por outra sequência numérica em formato string e terá o valor 2000. Neste caso, o valor 2000 será seguido de um ':' e pelos campos:

<apelido da origem>:<apelido do destino>:<controle de erro>:<CRC>:<mensagem>

Exemplo:

```
2000:B:A:maquinainexistente:19385749:Oi pessoal!
```

Funcionamento do pacote de dados

No destino

Ao receber um pacote de dados, a estação identifica se o mesmo é endereçado a ela, verificando o apelido do destino. Caso não seja, este pacote deve ser enviado para a próxima máquina do anel. Caso o pacote seja para ela, a aplicação deve recalculer o CRC, imprimir o apelido da origem e a mensagem, e deve também enviar o pacote para a próxima máquina do anel, alterando o campo *maquinainexistente* para ACK ou NAK.

Na origem

Caso o pacote de dados seja recebido por quem o originou (o apelido de origem será igual ao seu apelido), será necessário verificar o controle de erro, pois a mensagem deu toda a volta no anel. Ao receber o pacote com o campo em *maquinainexistente* ou ACK, um *token* deve ser enviado para a próxima máquina do anel. Caso o pacote venha com NAK, o mesmo deve ser retransmitido apenas uma vez na rede, trocando o NAK por *maquinainexistente*, colocando a mensagem original sem erro e enviando a mensagem para a máquina a sua direita na próxima passagem do *token*.

Controle do Token

A máquina que gera o *token* a primeira vez também deve controlá-lo. Essa máquina irá verificar se o *token* está passando por ela dentro de um determinado tempo. Dois problemas podem ser detectados por essa estação:

1. o *token* não passa dentro de um tempo estipulado (*timeout*): um novo *token* deverá ser gerado, pois o mesmo foi perdido por uma estação do anel. Para tanto, deverá haver uma opção de retirada do *token* do anel por uma das máquinas; OU
2. um *token* passa por ela em um tempo menor que o tempo mínimo: neste caso há mais de um *token* circulando na rede e, portanto, o *token* deverá ser retirado da rede. Nesse caso, deverá haver uma opção de geração de *token*, para que seja possível gerar *tokens* por qualquer máquina quando a rede estiver em funcionamento.

Alteração topológica do anel

A aplicação deverá permitir a alteração da topologia lógica do anel durante sua execução, possibilitando a entrada de máquinas sem a necessidade de reinicialização do sistema. Uma nova máquina pode entrar na rede somente quando dados não estiverem sendo enviados, ou seja, quando somente haverá o *token* circulando na rede.

Quando uma nova máquina entrar no anel, ela deve enviar a mensagem de DISCOVER e as demais máquinas devem responder com HELLO. A partir do HELLO, as máquinas devem utilizar a nova configuração do anel.

A demonstração deverá acontecer, no mínimo, em 3 máquinas.

Deve ser possível:

- Especificar, a qualquer momento:
 - uma mensagem a ser enviada por uma máquina;
 - a retirada de um *token* da rede;
 - a inclusão de um *token* na rede.
- Visualizar onde o *token* e o pacote de dados se encontram durante a execução do programa.
- Avisar quando houver retransmissões.
- Saber o que está acontecendo no anel (o que cada máquina está fazendo).
- Saber se houve *token* perdido ou se há mais de um *token* circulando na rede.

- Detecção de falhas e recuperação
 - Inclusão de novas máquinas no anel a qualquer momento.
-

Regras Gerais

Grupos: Até 4 componentes.

Data de entrega e apresentação: 22/06

Obs.: Todos os participantes devem estar presentes

Entrega final:

- Relatório descrevendo a estrutura da solução dada, envolvendo estruturas de dados, threads, classes, mecanismos de sincronização utilizados, CRC, exemplos de execução, etc.
- Código fonte comentado.

IMPORTANTE: Não serão aceitos trabalhos entregues fora do prazo. Trabalhos que não compilam ou que não executam não serão avaliados. Todos os trabalhos serão analisados e comparados. Caso seja identificada cópia de trabalhos, todos os trabalhos envolvidos receberão nota ZERO.